Securing Mobile Infrastructures against Privacy Attacks

Sanket Goutam

Presentation at Samsung Research America
Date: August 21, 2025

Research Motivation

- **Beyond phones** → wearables, AR/VR, IoT in smart homes
- **User-first design** → intimate, always-on, body & environment facing
- **Continuous sensing** → location, biometrics, gaze, surroundings
- **High privacy risk** → surveillance, profiling, misuse of personal context
- **Criticality** → protect users, not just devices

Stony Brook University

Erebus: Access Control for Augmented Reality Systems

Sanket Goutam*, Yoonsang Kim*,
Amir Rahmati, Arie Kaufman

** equal contribution*

Two form factors for building AR Systems

Standalone

Oculus Quest 2

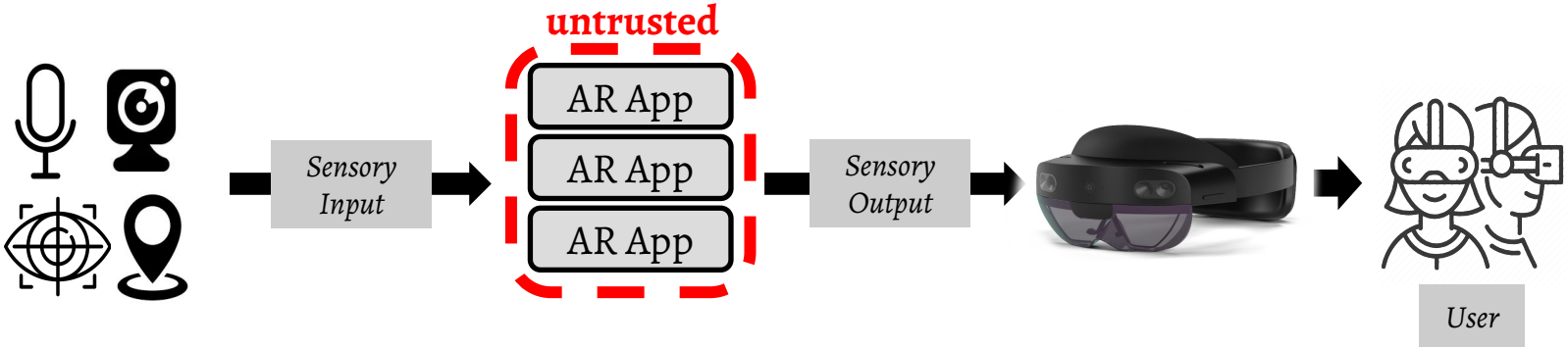
HoloLens 2

Companion Device

Rokid Air Pro

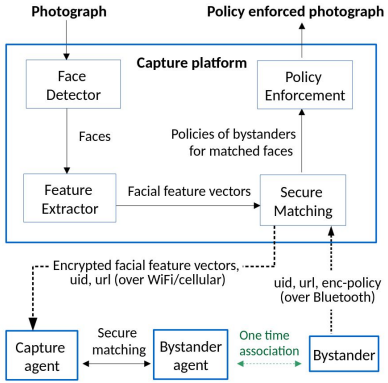
Toshiba dynaEdge

Applications derive information from device sensors.



Application Pipeline

Prior approaches — least privilege access



I-pic (Aditya et.al, ACM MobiSys'16)

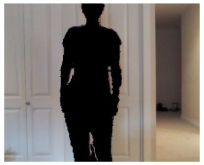
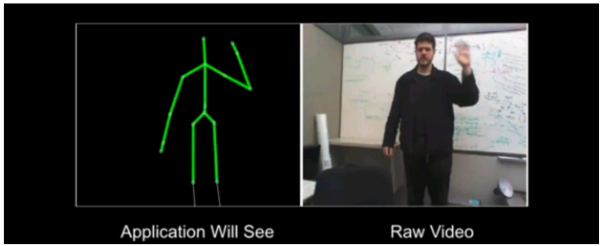


Figure 2: **World-Driven Access Control in Action.** *World-driven access control helps a user manage application permission to both protect her own privacy and to honor a bystander's privacy wishes. Here, a person asks not to be recorded; our prototype detects and applies the policy, allowing applications to see only the modified image. In this case, the person wears a QR code to communicate her policy, and a depth camera is used to find the person, but a variety of other methods can be used.*

Privacy Passports (Roesner et.al, ACM CCS'14)

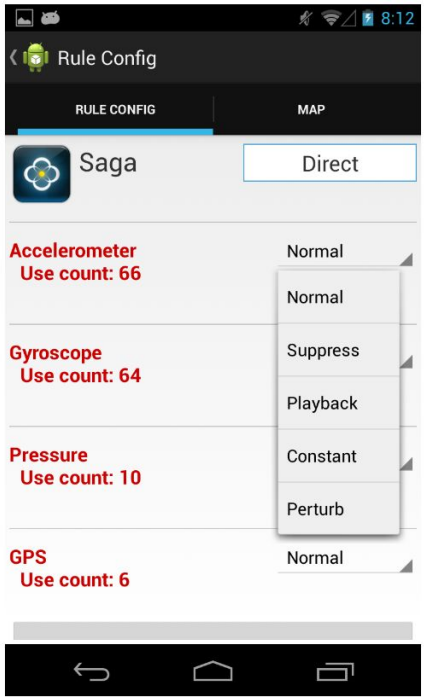
(a) (b)

PrivateEye and WaveOff (Raval et.al, ACM MobiSys'16)

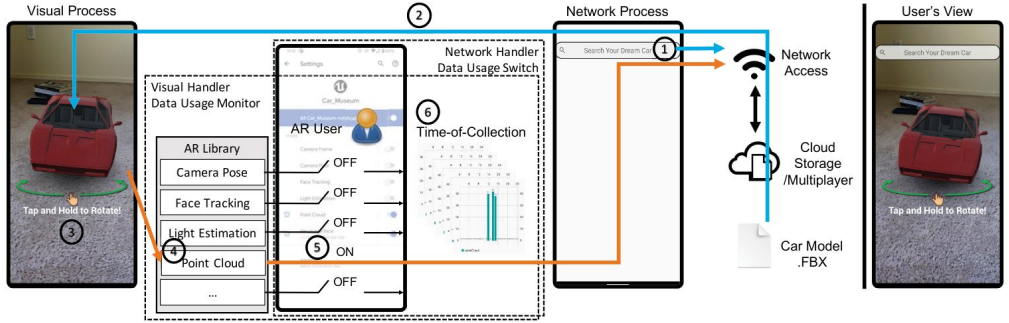


Recognizers (Jana et.al, USENIX Security'13)

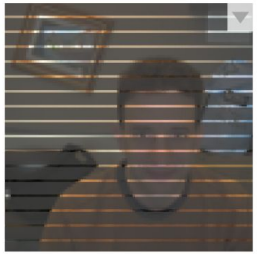
Prior approaches — user informed risk mitigation



ipShield (Chakraborty et.al, NSDI'14)



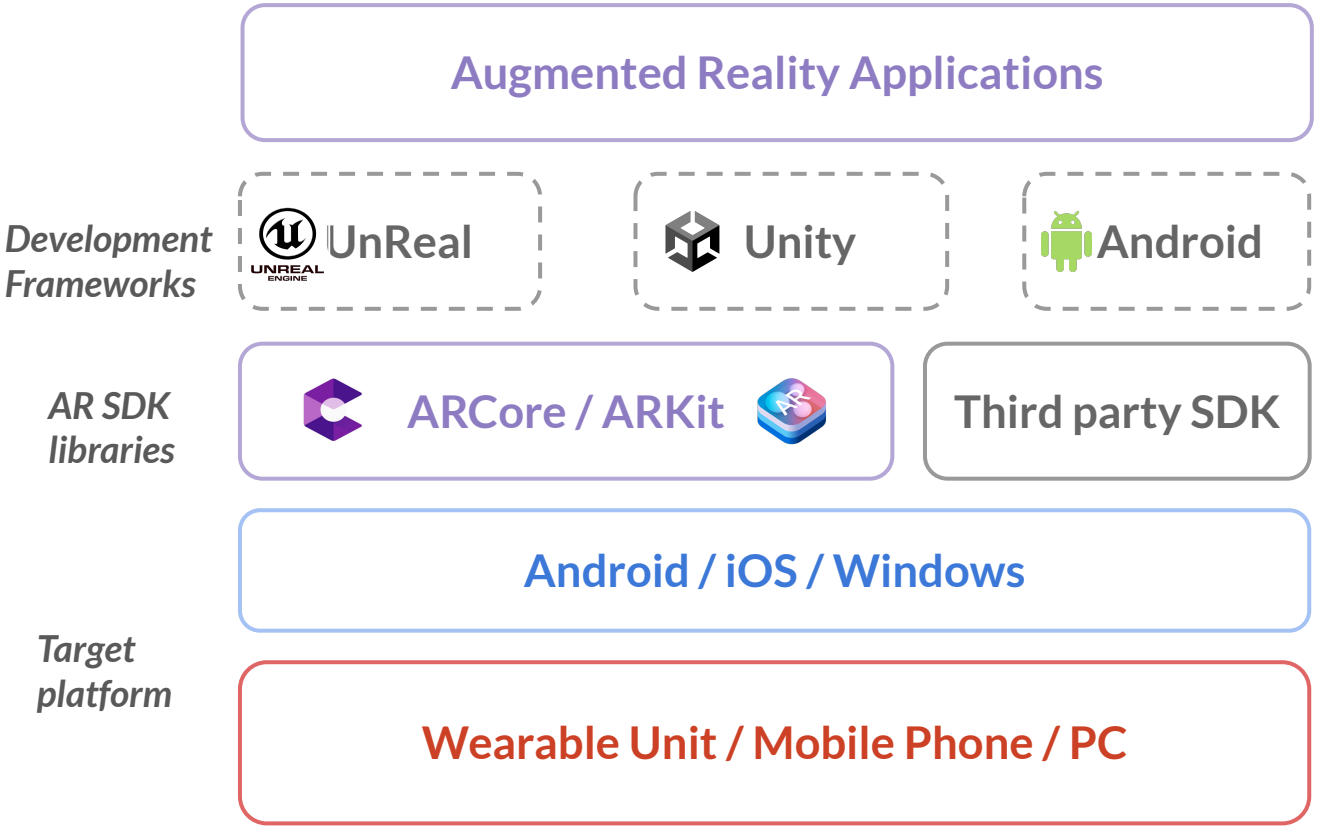
Lenscap (Hu et.al, ACM MobiSys'21)



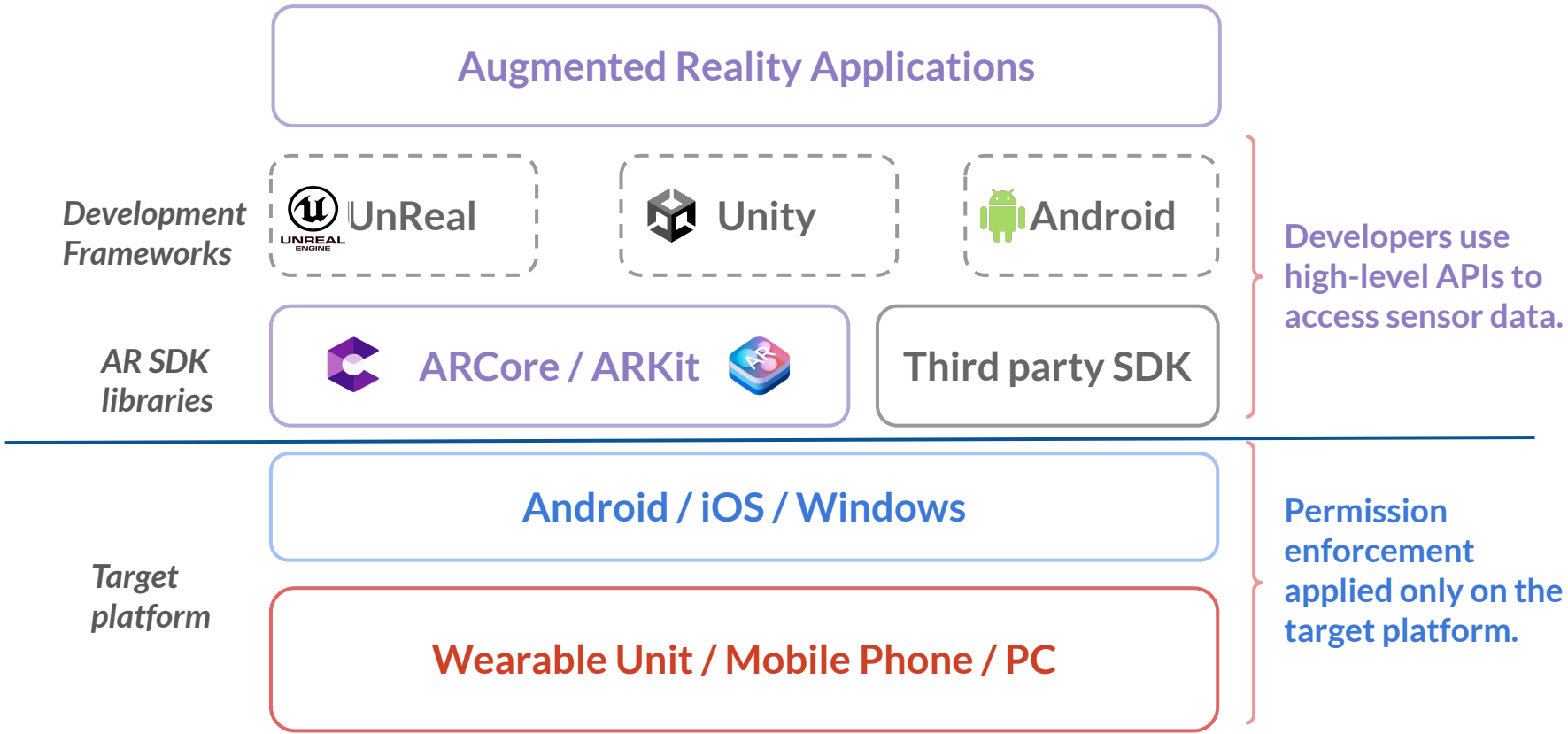
Access Gadgets (Howell and Schechter, IEEE Web S&P Workshop, 2010)

Figure 7: Window blinds illustrate that applications cannot see a data stream.

How are these applications developed today?



Dichotomy between data required and access requested.

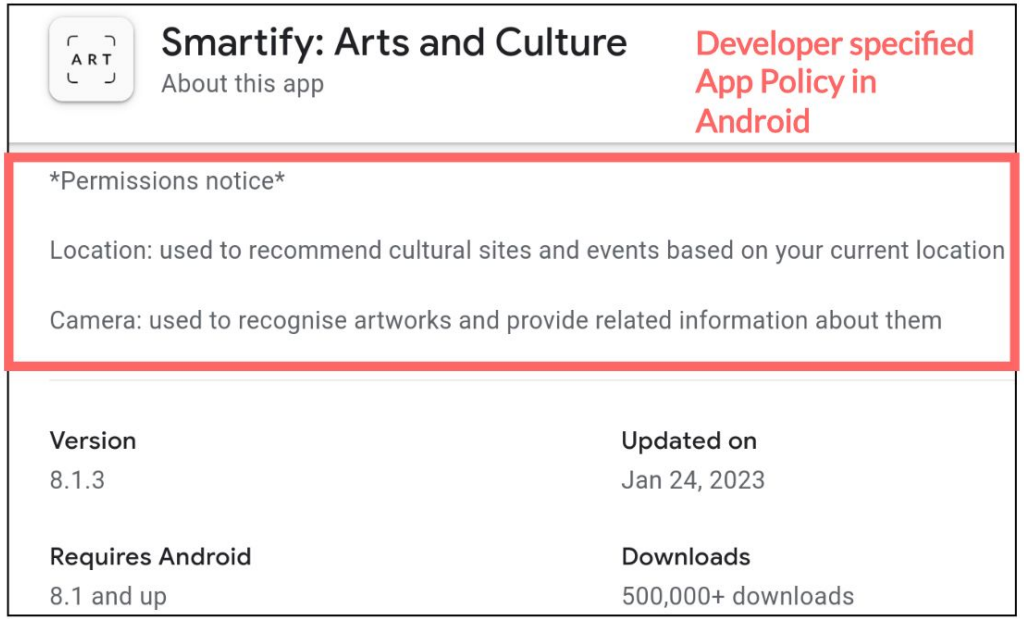


Permission Control similar to Smartphone OS.

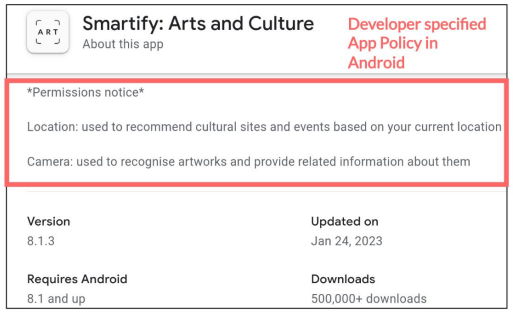
AR Device Type	Device Name	Platform	Access Control Mechanism
Standalone Wearable	Meta Quest 2 [43]	Android	App Manifest
	Microsoft HoloLens 2 [44]	Windows	App Manifest, Policy CSP
	Magic Leap 2 [16]	And	
	Google Glass Enterprise [23]	A	
	ThirdEye X2 MR Smart Glasses [22]	A	
	Vuzix Blade AR [70]	A	
	Snap Spectacles [67]	A	
	Raptor AR Headset [19]	A	
	Kopin Solos [36]	A	
	Xiaomi Smart Glasses [68]	A	
With a Companion Device	Lenovo ThinkReality A3 [40]	A	
	Epson Moverio [18]	And	
	Toshiba dynaEdge [63]	Windows	mechanism
	Rokid Air Pro [52]	Android, iOS	App Manifest
	NReal Light [46]	Android	No AC mechanism
	Viture One [69]	Android	No information available
	Dream Glass Flow [66]	Android, iOS	No information available

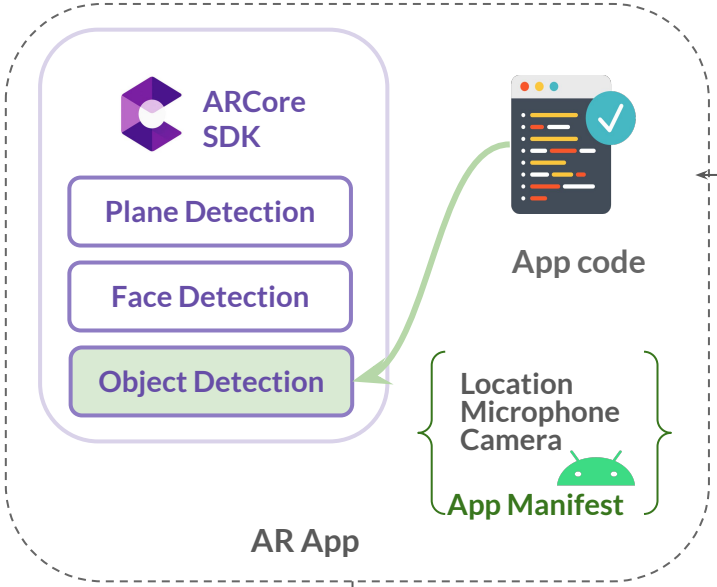
```
<uses-feature android:name="android.hardware.camera"
  android:required="true" />
<uses-permission android:name="android.permission.record_audio"
  android:required="true" />
<uses-feature android:name="android.hardware.location.GPS"
  android:required="true" />
<uses-feature android:name="android.hardware.sensor.heartrate"
  android:required="true" />
```

Developer specifies an access policy to user on Play Store.



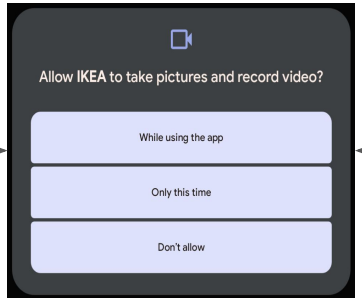
User installs the app on their device.





Access sensor data.

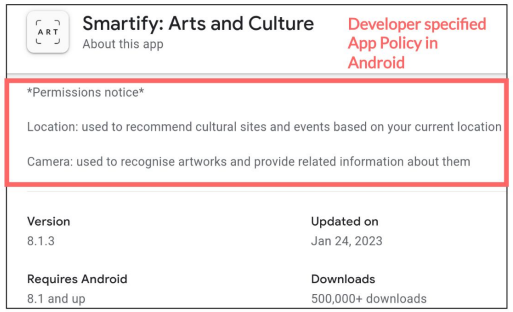
Device Sensors

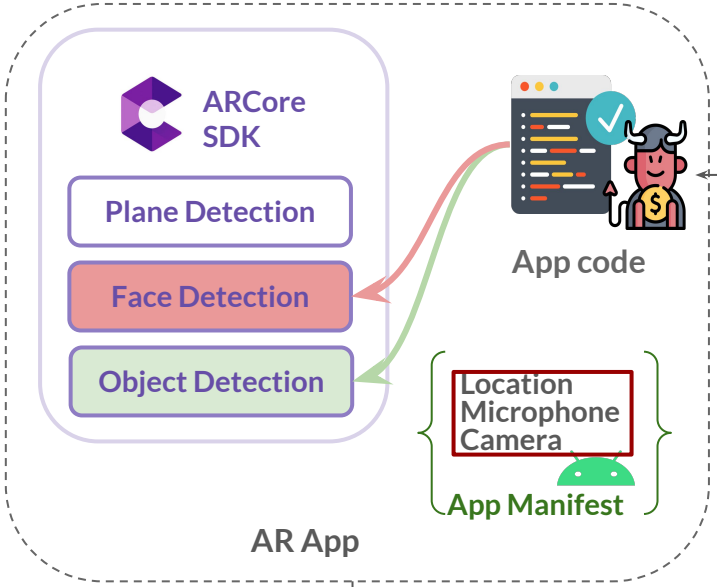


Prompt user to grant access to sensors.

Grants permissions on first use.

Malicious app can violate app policy.

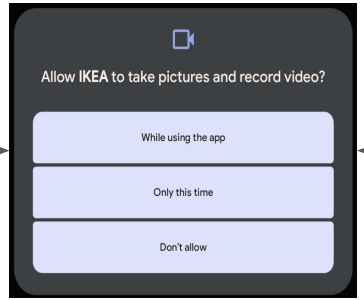




App code

Access sensor data.

Device Sensors



Prompt user to grant access to sensors.

Grants permissions on first use.

Can we reimagine Access Control for visionOS?

SPATIAL COMPUTING —

Unity's visionOS support has started to roll out—here's how it works

A closed beta will admit developers gradually over the coming weeks.

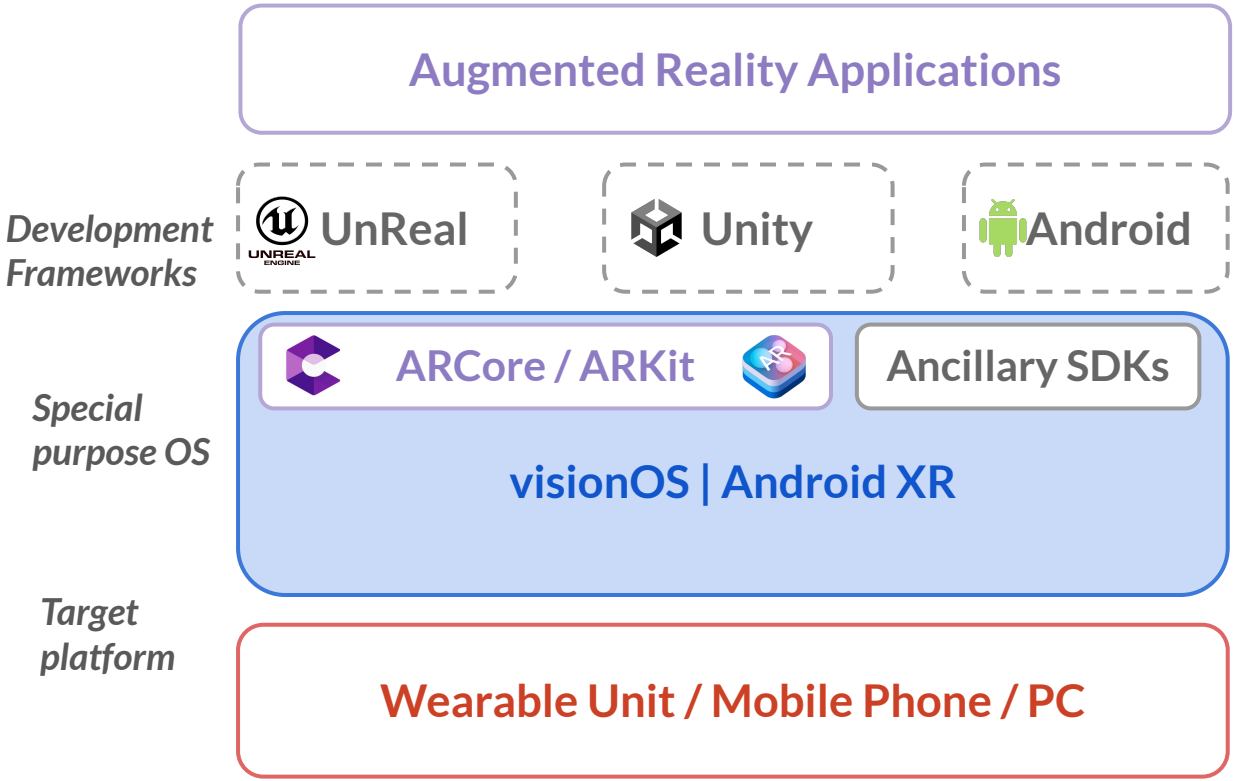
SAMUEL AXON - 7/19/2023, 3:51 PM

G1. How can we regulate direct access to sensors?

G2. How to ensure a least privilege access based on developer-specified policy, allowing access to what's required and nothing more?

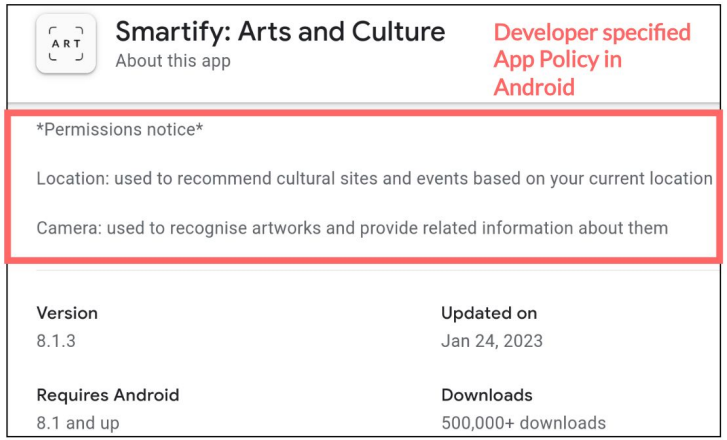
G3. Can we allow users to adjust access based on their requirements?

Erebus: regulating sensor access at the OS-level



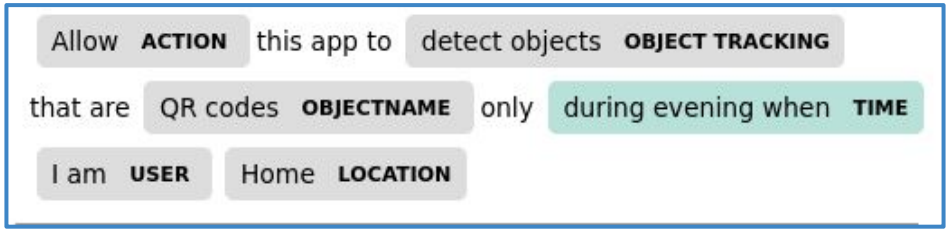
Integrating AR functional requirements with permission control at the OS-level.

Erebus: policy specification language that *expresses* functionality

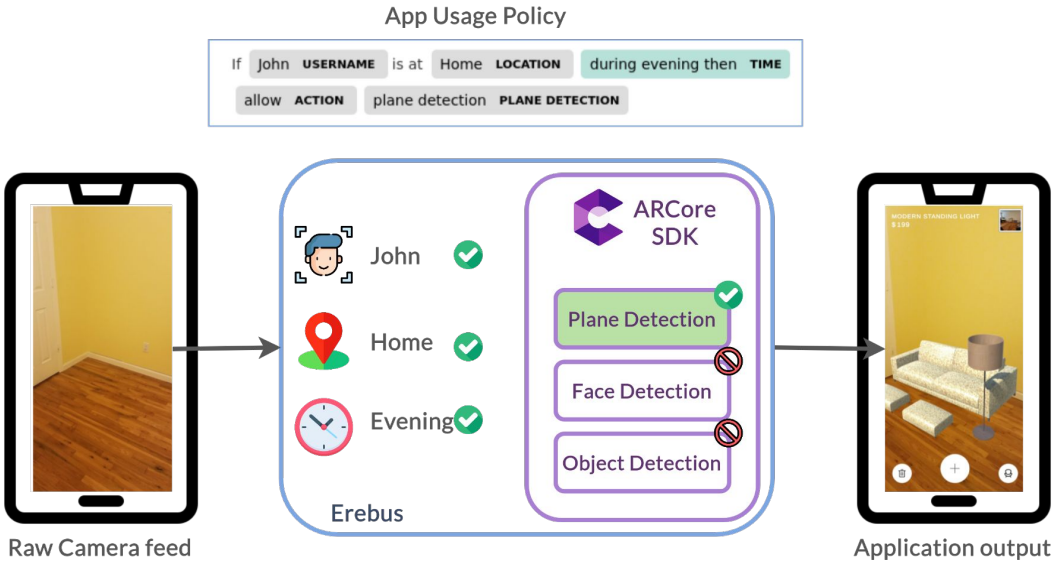


- Coarse-grained access requirement.
  (Location, Camera)
- Functional requirement cannot be enforced by the system
  (recognize artworks).

- Functional description in a semi-structured natural language format.
- Fine-grained permission enforcement.



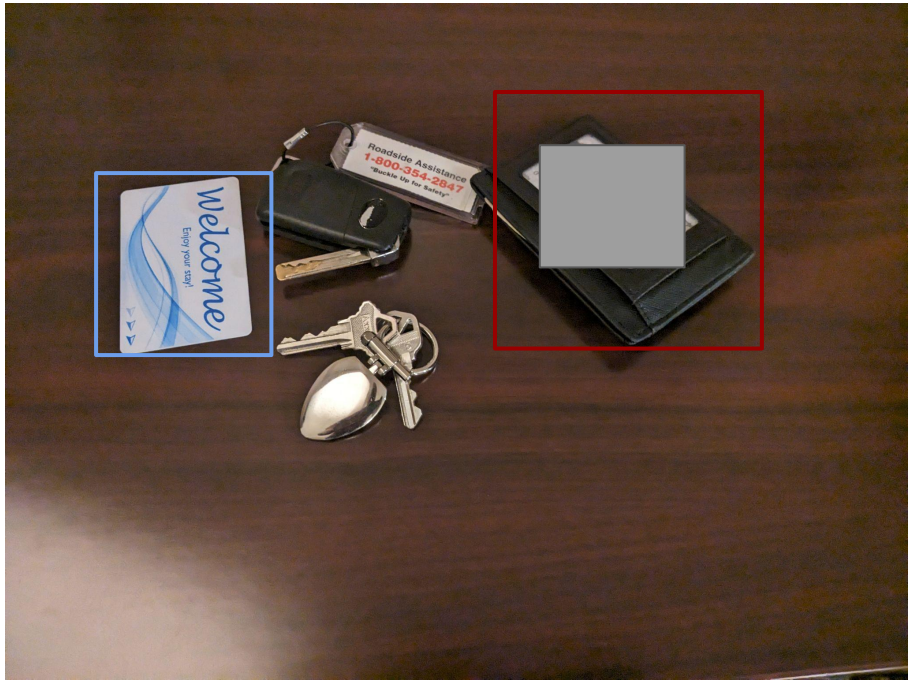
Erebus: users define *what*, *when*, *and* *where* data can be accessed



System validates app's sensor request based on context-dependent policy specification, ensuring ***least-privilege.***

Erebus: preventing sensitive data leakage

Object detection is an imperfect process. False positives could leak sensitive information to the app.



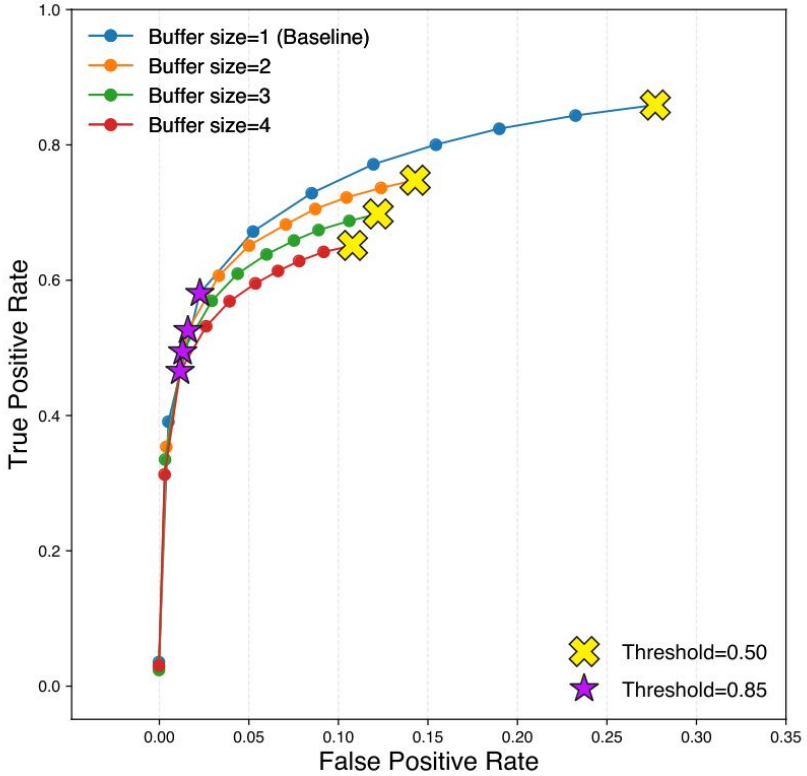
How does Erebus prevent leaking sensitive data due to false positives?

We leverage ***conflation*** technique to optimize object detection accuracy and reduce false positives in Erebus.

How to Combine Independent Data Sets for the Same Quantity

By

Theodore P. Hill[1] and Jack Miller[2]



Does Erebus incur additional latency over API calls?

API Type	Erebus (ms)	Unprotected (ms)
Camera sensor-based API	0.35 ± 0.12	0.18 ± 0.04
Location sensor-based API	0.22 ± 0.04	<0.01

By enforcing runtime checks on API calls, there is a small overhead incurred by Erebus but this has no impact on performance.

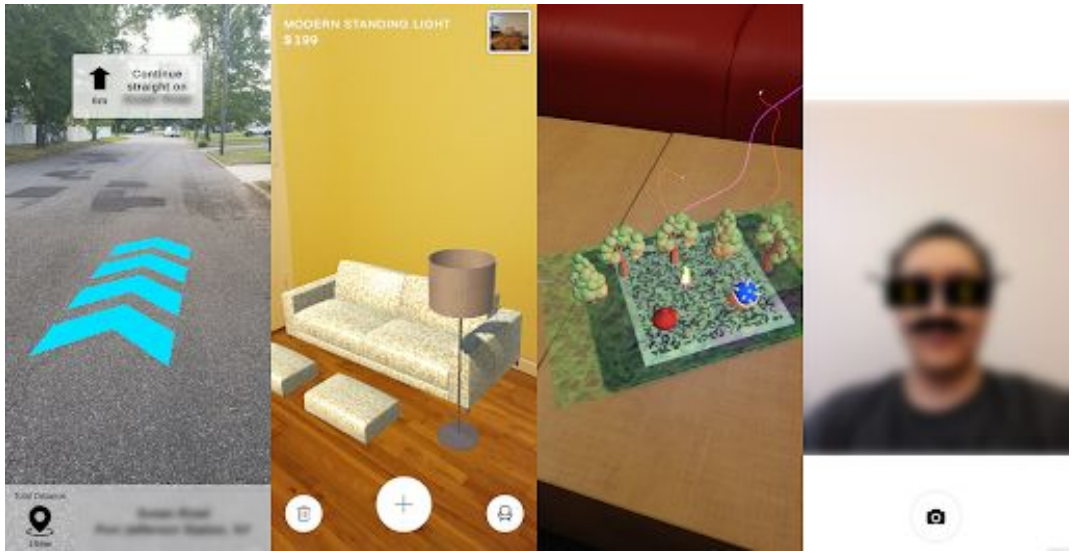
Does Erebus affect app's overall performance?

Component Type	Component	Latency (ms)
Erebus	Object Detection	28.91
	Non-Max Suppression	0.02
	Object Tracking	0.25
	Conflation	0.01
	Whitelisting	0.08
Application	Async GPU Readback (Constant)	181.72
	Application Logic	33.47
Overall Latency		**244.46**

Our prototype apps were able to run at **~34.16 FPS** with Erebus framework enforcing runtime checks.

Erebus: AR Access Control Framework

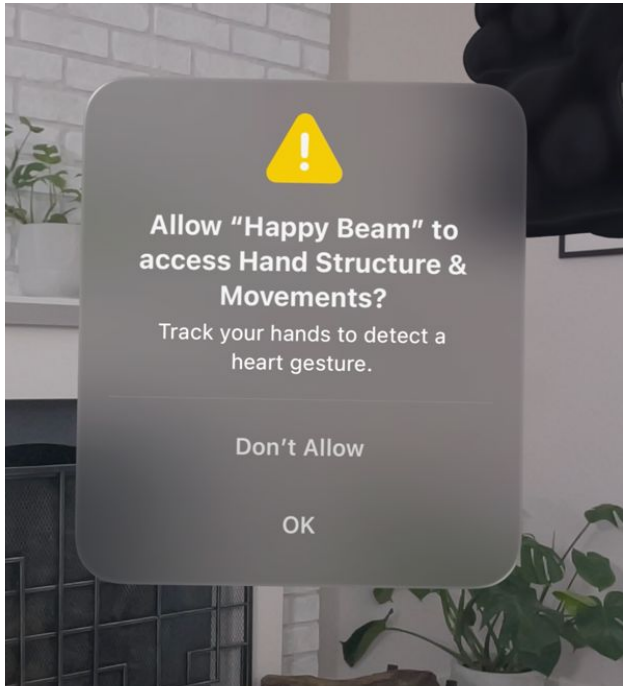
32ND USENIX SECURITY SYMPOSIUM



- Implemented on Google ARCore SDK using Unity Framework.
- Adapted 5 prototype AR applications to our framework.
- We open-source our framework implementation, policy-language design, and prototype applications for developer's reference.

Published in August 2023.

Erebus: Research Impact



Permissions

Some XR features require system permissions on the Android XR runtime. Refer to the following table for a list of OpenXR features and the permissions they require. Each feature is documented with a code example you can use to request the relevant permission.

Feature	Permission
Plane detection	android.permission.SCENE_UNDERSTANDING
Hand tracking	android.permission.HAND_TRACKING
Face tracking	android.permission.EYE_TRACKING
Gaze interaction profile	android.permission.EYE_TRACKING_FINE

VisionOS2 - June, 2024

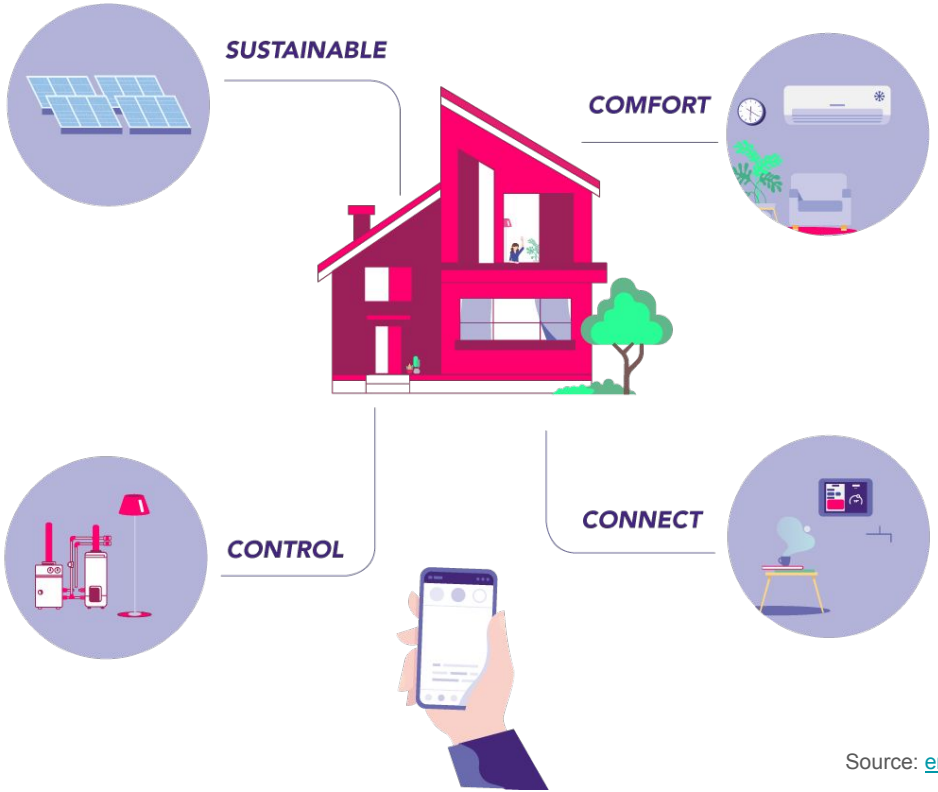
Unity OpenXR Android - December, 2024

Stony Brook University

Atlas: Cross-Vendor IoT Device-to-Device Authentication

Sanket Goutam, Omar Chowdhury, Amir Rahmati

Internet of Things (IoT) has revolutionized our homes.

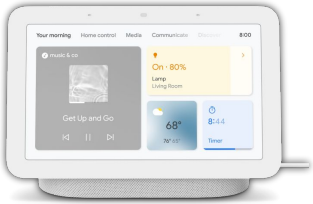


Source: [enelX Residential Solutions]

Devices interact to augment their capabilities.

Sensors

Actuators

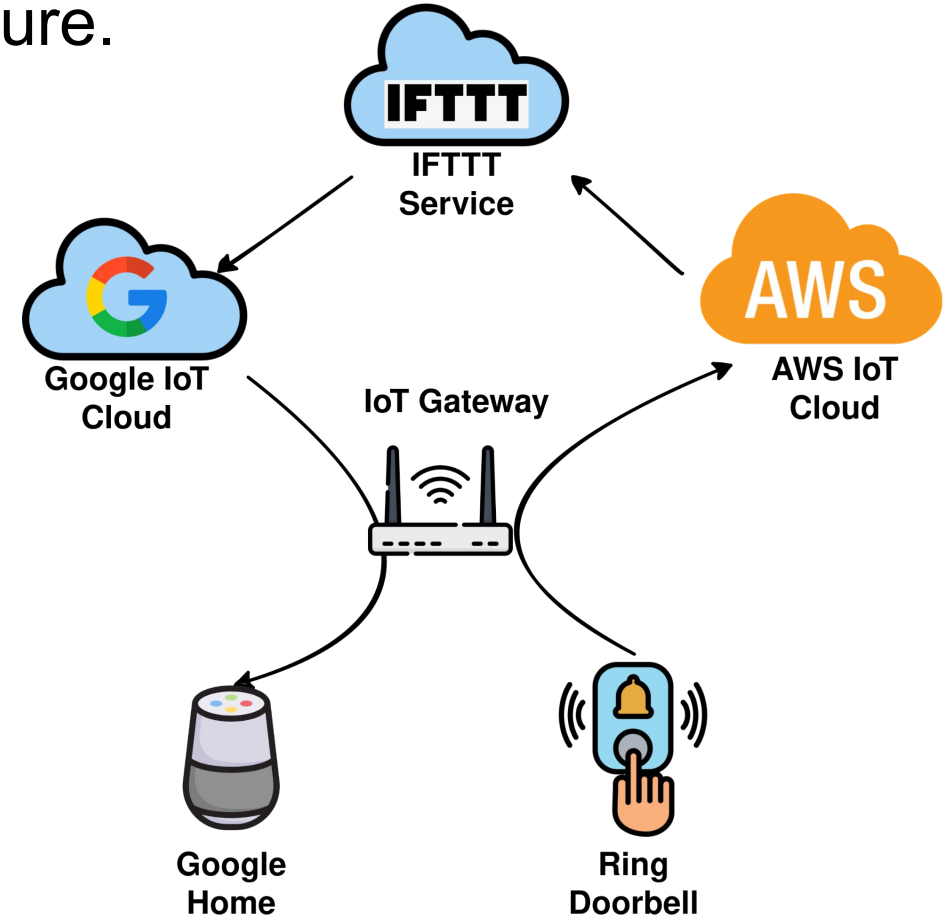


Smart Hubs

Source: [How to beat burglars at their own game]

It is a Cloud-first Infrastructure.

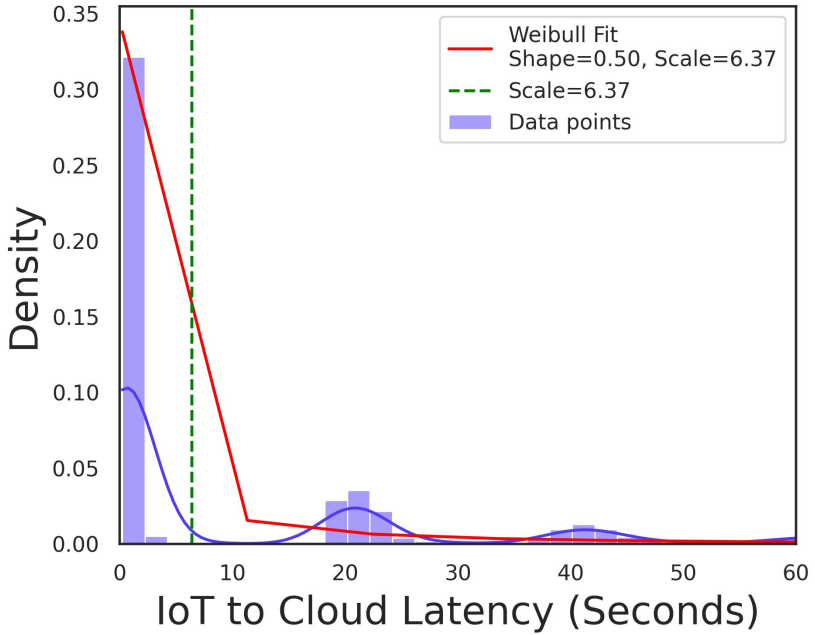
- Cross vendor interactions require a multi-cloud-hop model.
- Repeatedly found to be insecure and inadequate.



[1] Shattered chain of trust: Understanding security risks in cross-cloud iot access delegation," in 29th USENIX security symposium (USENIX security 20), 2020.
[2] "Data privacy in trigger-action systems," in 2021 IEEE Symposium on Security and Privacy (SP), 2021.
[3] Practical data access minimization in Trigger-Action platforms," in 31st USENIX Security Symposium (USENIX Security 22), 2022.

Cloud-first is also unfit for performance sensitive applications.

- Latency sensitive applications require 20-100 ms round trip latency.
- Cloud-first models require 0.3seconds latency in the average case.
- Performance drastically degrading under stress (as measured on AWS-IoT).



Why is it this way?

- Device authentication is important for vendors but also a hard problem.
- IoT devices have very diverse deployment schemes — some remain offline, some require real-time access, some are frequently moved between networks.
- Some IoT devices require constant cloud support while others require only local device support.
- These devices also have widely varying computational capabilities and manufacturing processes.

TABLE 1: Comparison of current relevant approaches for establishing device identity and authentication in IoT.
Pairing-based Encryption (PBE) establishes symmetric keys between devices through physical or contextual proximity. Identity-based Device Authentication (IDA) uses network behavior or hardware primitives to infer device identity, but **does not** support the full authentication lifecycle in the traditional sense. Blockchain-based PKI (b-PKI) decentralizes authentication by embedding device behavior and attestation records into distributed ledgers. Traditional PKI (PKI) solutions involve widely adopted X.509 digital certificates for device authentication. See §6.4 for detailed analysis.
○: No Support, ◐: Unlikely Adoption / Minimal Support, ●: Readily Supports, ✔: Currently Deployed

Method	System	Scalability	Cross-Vendor Interoperability	Legacy Support	Granular Revocation	Low Runtime Overhead	Decentralized Auth
PBE	IoTCupid [20]	○	●	◐	●	●	○
	UniverSense [24]	○	◐	◐	○	◐	○
	T2Pair [25]	○	●	◐	○	◐	○
	Move2Auth [26]	○	●	◐	○	◐	○

TABLE 1: Comparison of current relevant approaches for establishing device identity and authentication in IoT.
Pairing-based Encryption (PBE) establishes symmetric keys between devices through physical or contextual proximity. Identity-based Device Authentication (IDA) uses network behavior or hardware primitives to infer device identity, but **does not** support the full authentication lifecycle in the traditional sense. Blockchain-based PKI (b-PKI) decentralizes authentication by embedding device behavior and attestation records into distributed ledgers. Traditional PKI (PKI) solutions involve widely adopted X.509 digital certificates for device authentication. See §6.4 for detailed analysis.
○: No Support, ◐: Unlikely Adoption / Minimal Support, ●: Readily Supports, ✔: Currently Deployed

Method	System	Scalability	Cross-Vendor Interoperability	Legacy Support	Granular Revocation	Low Runtime Overhead	Decentralized Auth
PBE	IoTCupid [20]	○	●	◐	●	●	○
	UniverSense [24]	○	◐	◐	○	◐	○
	T2Pair [25]	○	●	◐	○	◐	○
	Move2Auth [26]	○	●	◐	○	◐	○
IDA	IoT-ID [27]	○	◐	◐	●	◐	◐
	MCU-Token [22]	○	◐	◐	●	◐	◐
	Z-IoT [28]	○	○	◐	●	○	○
	IoT-Sentinel [29]	○	◐	●	●	○	○

TABLE 1: Comparison of current relevant approaches for establishing device identity and authentication in IoT.
Pairing-based Encryption (PBE) establishes symmetric keys between devices through physical or contextual proximity. Identity-based Device Authentication (IDA) uses network behavior or hardware primitives to infer device identity, but **does not** support the full authentication lifecycle in the traditional sense. Blockchain-based PKI (b-PKI) decentralizes authentication by embedding device behavior and attestation records into distributed ledgers. Traditional PKI (PKI) solutions involve widely adopted X.509 digital certificates for device authentication. See §6.4 for detailed analysis.
○: No Support, ◐: Unlikely Adoption / Minimal Support, ●: Readily Supports, ✔: Currently Deployed

Method	System	Scalability	Cross-Vendor Interoperability	Legacy Support	Granular Revocation	Low Runtime Overhead	Decentralized Auth
PBE	IoTCupid [20]	○	●	◐	●	●	○
	UniverSense [24]	○	◐	◐	○	◐	○
	T2Pair [25]	○	●	◐	○	◐	○
	Move2Auth [26]	○	●	◐	○	◐	○
IDA	IoT-ID [27]	○	◐	◐	●	◐	◐
	MCU-Token [22]	○	◐	◐	●	◐	◐
	Z-IoT [28]	○	○	◐	●	○	○
	IoT-Sentinel [29]	○	◐	●	●	○	○
b-PKI	Academic Proposals [30, 31, 32]	◐	●	○	◐	○	●
	Industry Proposal (Knox Matrix [33])	◐	●	○	◐	○	●

TABLE 1: Comparison of current relevant approaches for establishing device identity and authentication in IoT.
Pairing-based Encryption (PBE) establishes symmetric keys between devices through physical or contextual proximity. Identity-based Device Authentication (IDA) uses network behavior or hardware primitives to infer device identity, but **does not** support the full authentication lifecycle in the traditional sense. Blockchain-based PKI (b-PKI) decentralizes authentication by embedding device behavior and attestation records into distributed ledgers. Traditional PKI (PKI) solutions involve widely adopted X.509 digital certificates for device authentication. See §6.4 for detailed analysis.
○: No Support, ◐: Unlikely Adoption / Minimal Support, ●: Readily Supports, ✔: Currently Deployed

Method	System	Scalability	Cross-Vendor Interoperability	Legacy Support	Granular Revocation	Low Runtime Overhead	Decentralized Auth
PBE	IoTCupid [20]	○	●	◐	●	●	○
							○
							○
							○
IDA							◐
							◐
							○
							○
b-PKI							●
	(Knox Matrix [33])						●
PKI	Vendor-controlled PKI [34, 35, 36] ✔	○	○	●	○	●	○
	Alliance-based PKI [7, 10, 2] ✔	◐[2]	◐[2]	●	◐[2]	◐[2]	◐[2]

IoT market fragmentation complicates device deployment

Increasingly niche IoT offerings with inconsistent interoperability practices have led to a fragmented IoT market to the detriment of easy IoT adoption.

By **Mary E. Shacklett**, Transworld Data

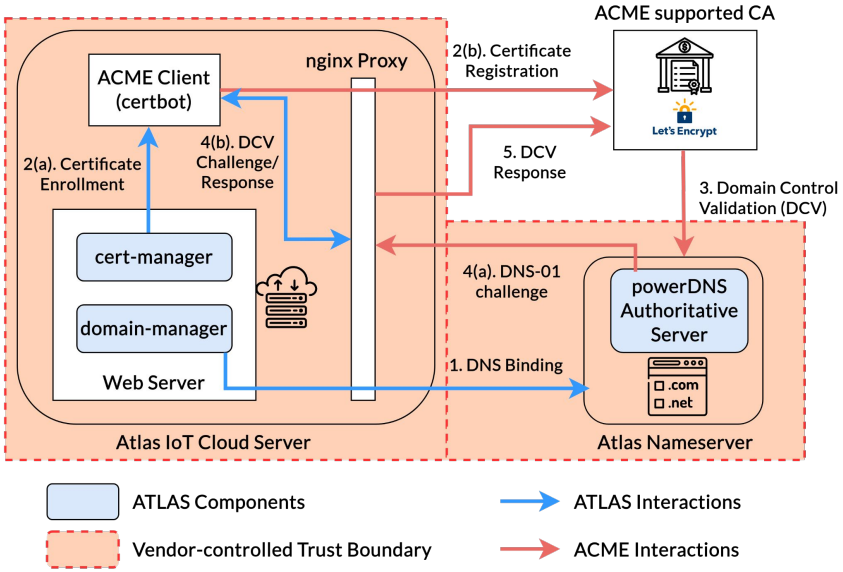
Published: **08 Feb 2021**

PKI for IoT – Challenges

- IoT devices are not directly exposed to the Internet.
  - They sit behind network gateways (NAT) and only have locally unique IDs (MAC address).
- Most of them even remain offline.
  - OTA certificate renewals are not possible, so they just use self-signed wildcard certificates.
- Web certificates are issued to servers by verifying their ownership of a web-based entity (DCV - Domain Control Validation).
  - If you own the rights to your own domain name, and have a public DNS entry pointing to your web server's public IP address.
  - This is not possible for IoT devices. Exposing IoT devices to the public Internet would make them easy targets *(Mirai Botnet)*.

Key Insight: Adapt ACME Model for IoT

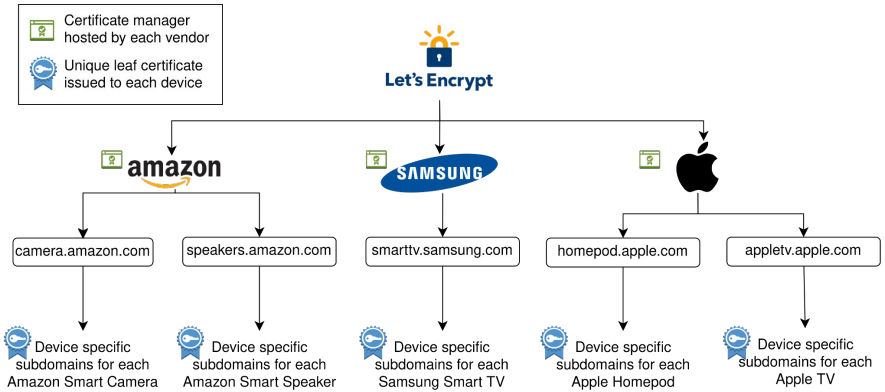
- ACME (Automated Certificate Management Environment), popularized by Lets Encrypt, allows for free and automated certificate management for the Web.
- Formally verified to be secure.
- Designed for Web scale, Lets Encrypt alone issues 600M certificates a day.



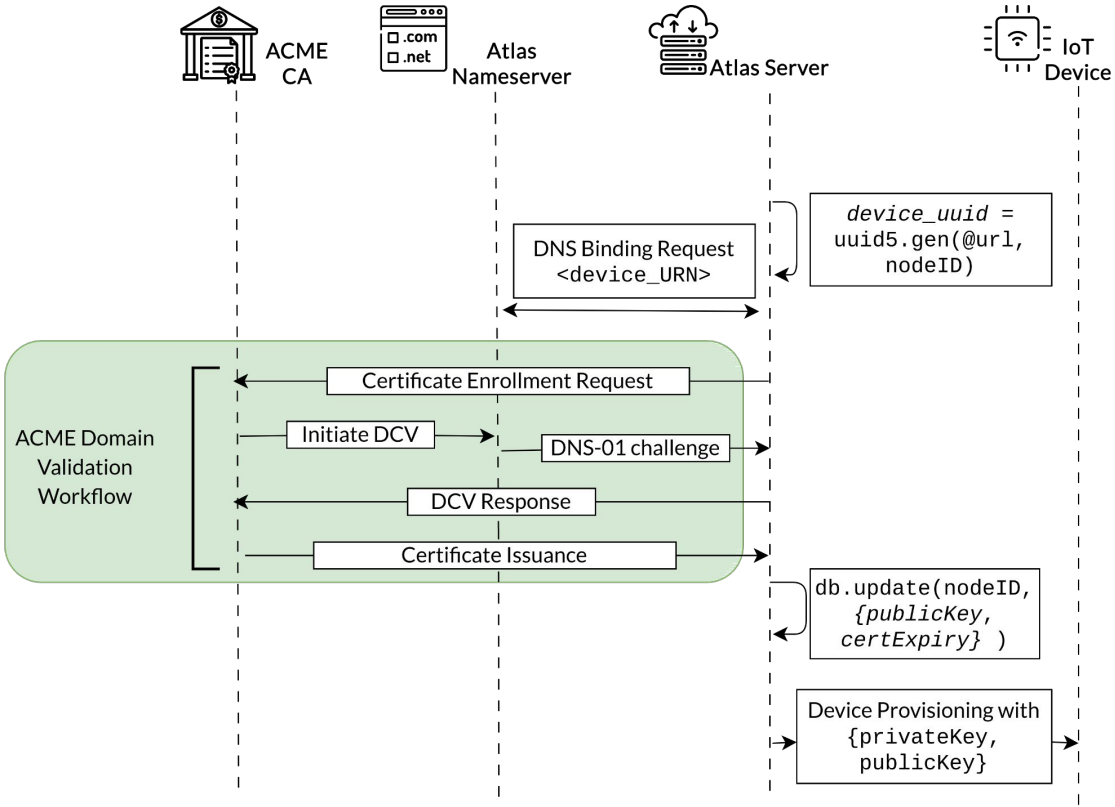
Atlas: Extending ACME with IoT Namespaces

TABLE 2: An example Namespace for IoT devices. *Note that, ATLAS's design is parametric to the choice of namespace* and can support other vendor-specific customization.

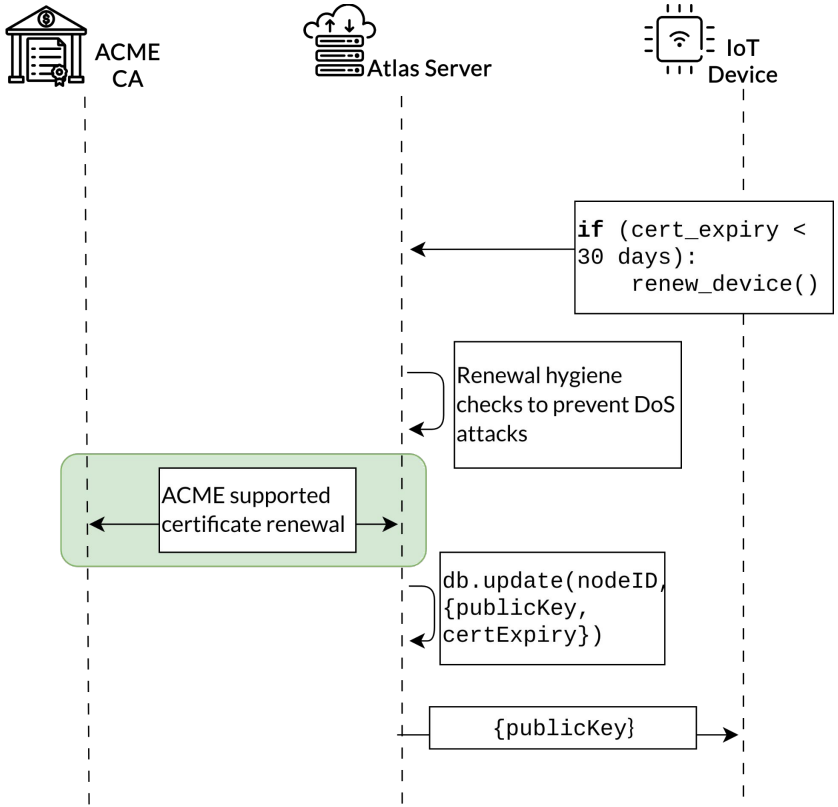
⟨URN⟩	⊨	⟨UUID⟩.⟨device-class⟩.⟨root-domain⟩
⟨UUID⟩	⊨	4⟨hexOctet⟩-2⟨hexOctet⟩- 2⟨hexOctet⟩-2⟨hexOctet⟩- 6⟨hexOctet⟩
⟨hexOctet⟩	⊨	⟨hexDigit⟩⟨hexDigit⟩
⟨hexDigit⟩	⊨	⟨digit⟩ | A | B | C | D | E | F
⟨digit⟩	⊨	%x30-39
⟨device-class⟩	⊨	*Vendor-specific categorization of the device,* e.g., *'smart-speaker', 'smart-hub', etc.*
⟨root-domain⟩	⊨	*The root domain of the vendor,* e.g., *'amazon.com', 'ring.com', etc.*



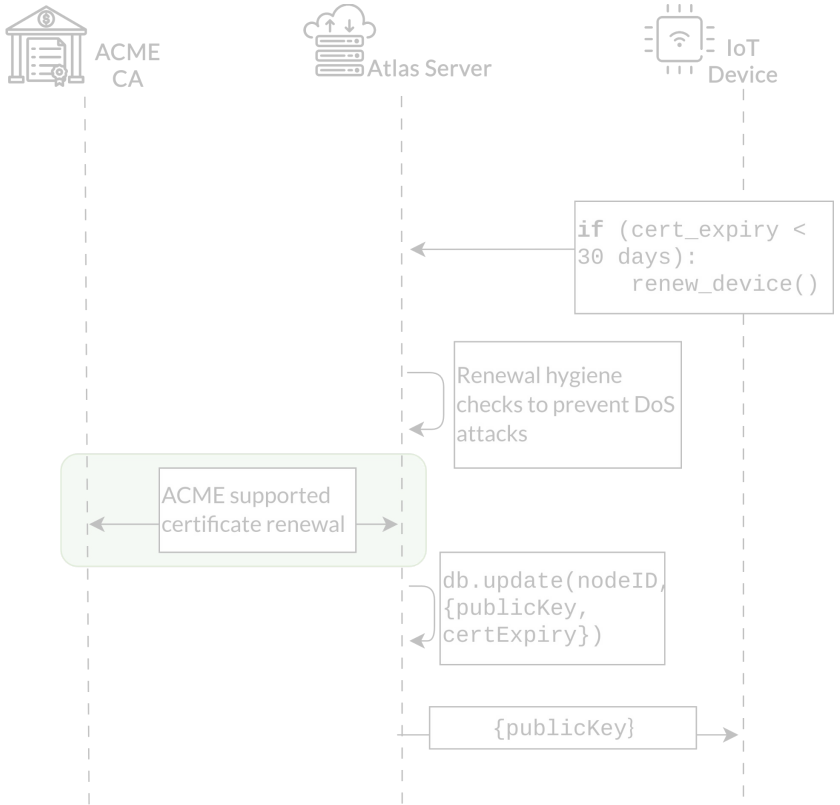
Atlas: Binding and Enrollment step

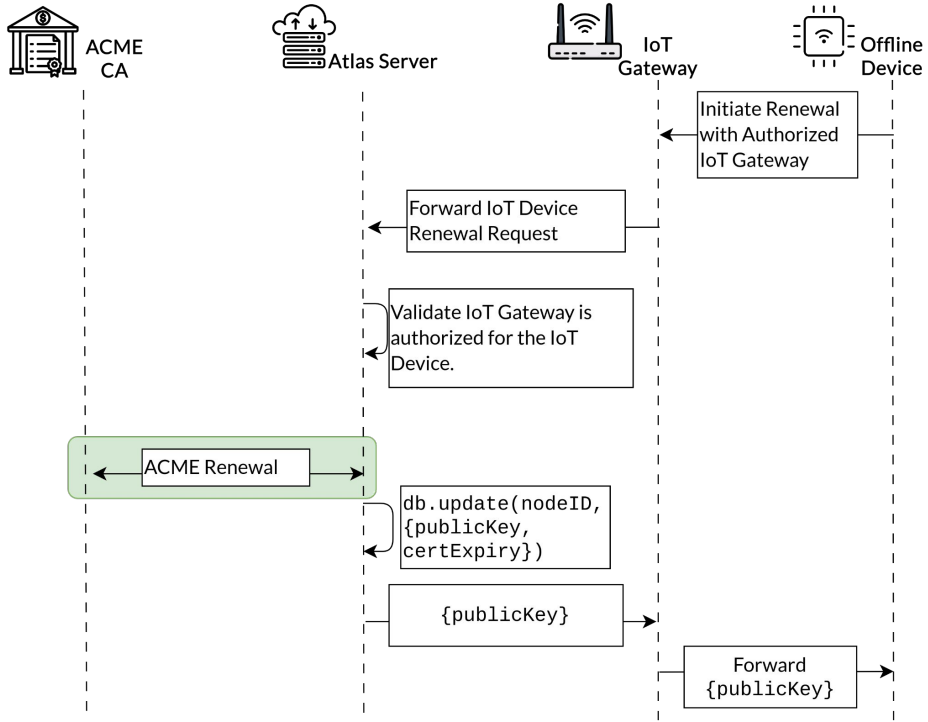


Atlas: Automatic Renewal of Online IoT Device

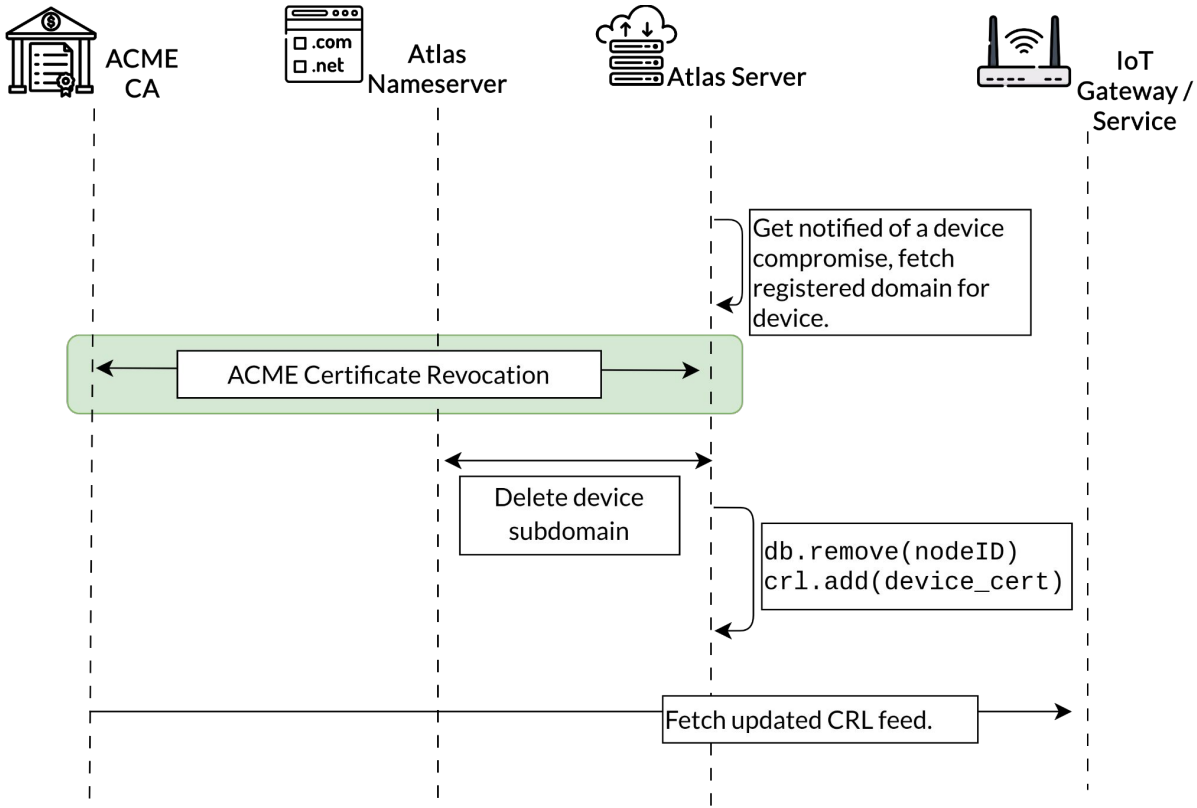


Atlas: Automatic Renewal of Offline IoT Device



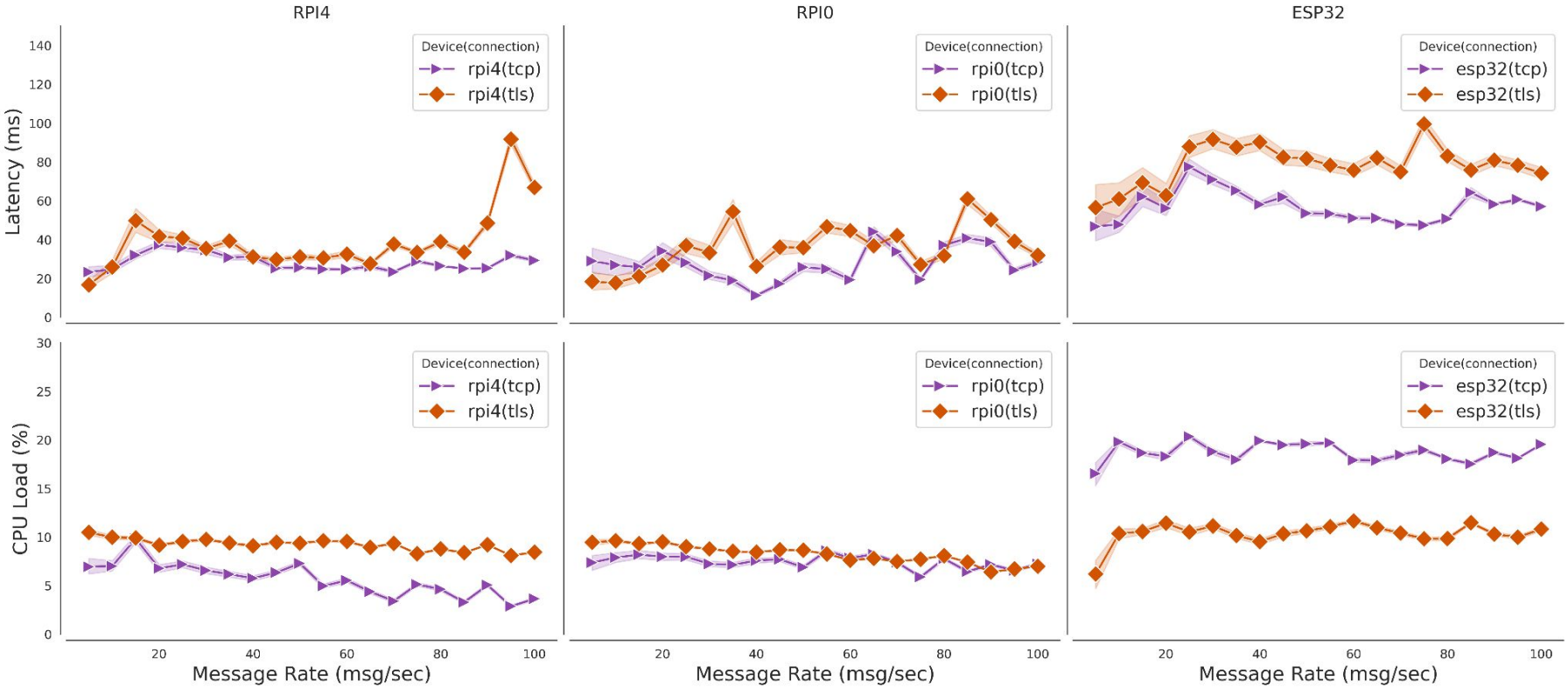


Atlas: Fine-grained Revocation of Compromised Devices



Atlas: Enforces mutual-TLS for Cross-Vendor Interaction

Is mutual-TLS practical in IoT?



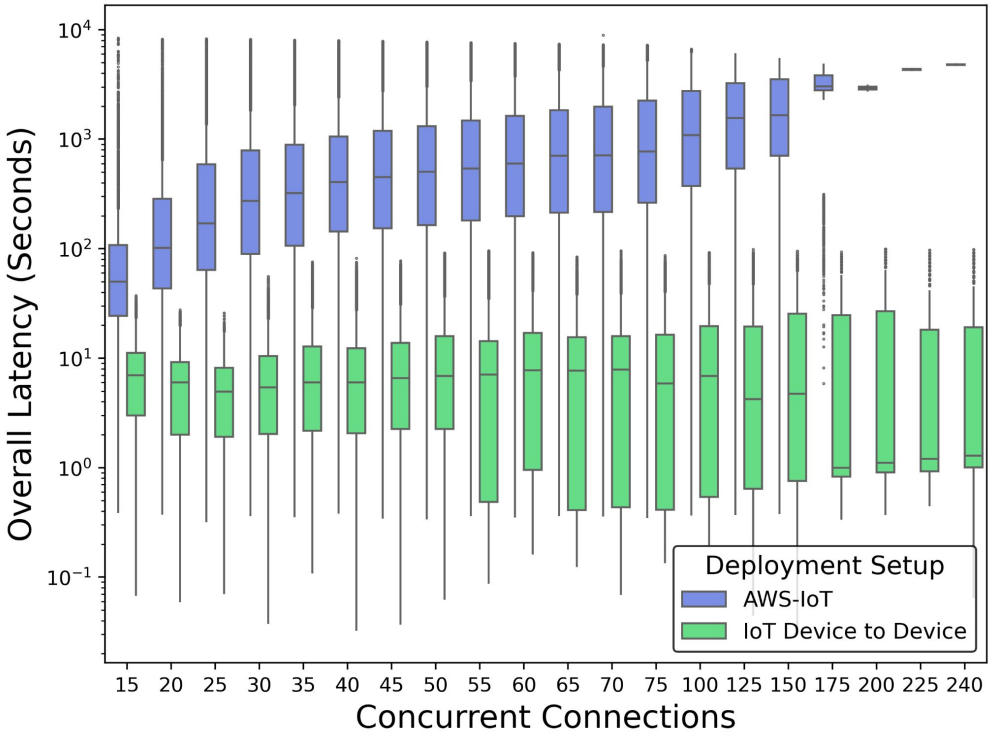
Modern microcontrollers are highly performant for TLS.

TABLE 3: Latency measurements show that the overhead of TLS is well within the real-time performance thresholds for IoT. Modern micro-controllers (*e.g.*, esp32) have dedicated cryptographic accelerators that offload all TLS computation, which is why our load measurement on general purpose CPU for TLS in lower than TCP.

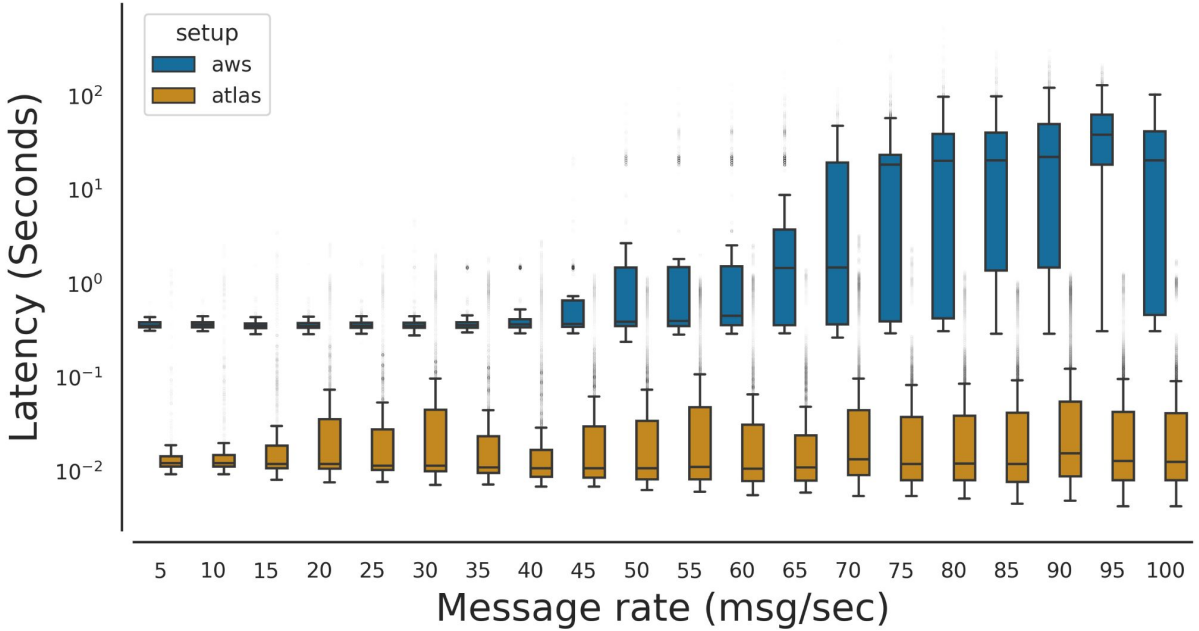
Metric		rpi4	rpi0	esp32
Latency (ms)	tcp	27.678543	29.253048	56.541336
	tls	44.278107	39.473658	80.758832
CPU Usage (%)	tcp	4.802685	7.282358	18.690758
	tls	8.943186	7.691518	10.557885

Comparison against cloud-first setups.

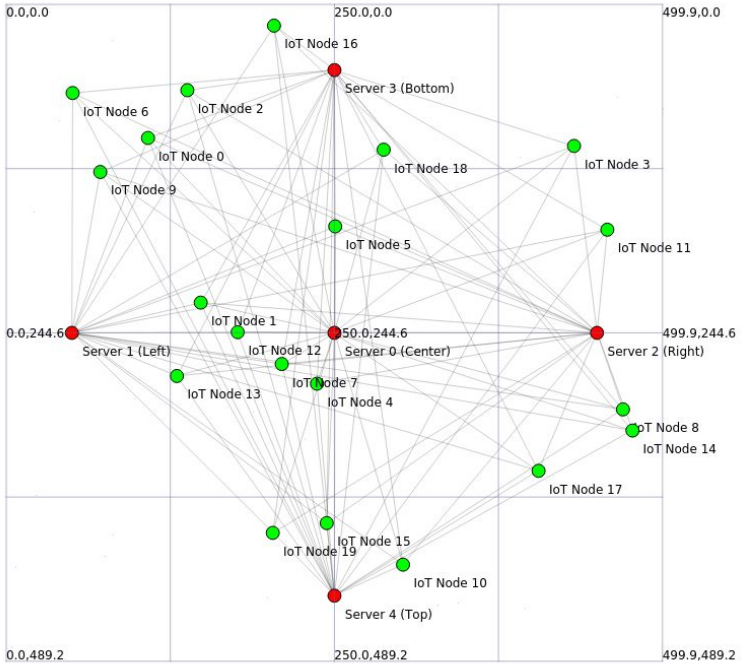
IoT Clouds (AWS-IoT) start rate limiting applications under stress (>100 connections), resulting in lower throughput due to packet loss, despite claiming to support higher limits (250 connections).

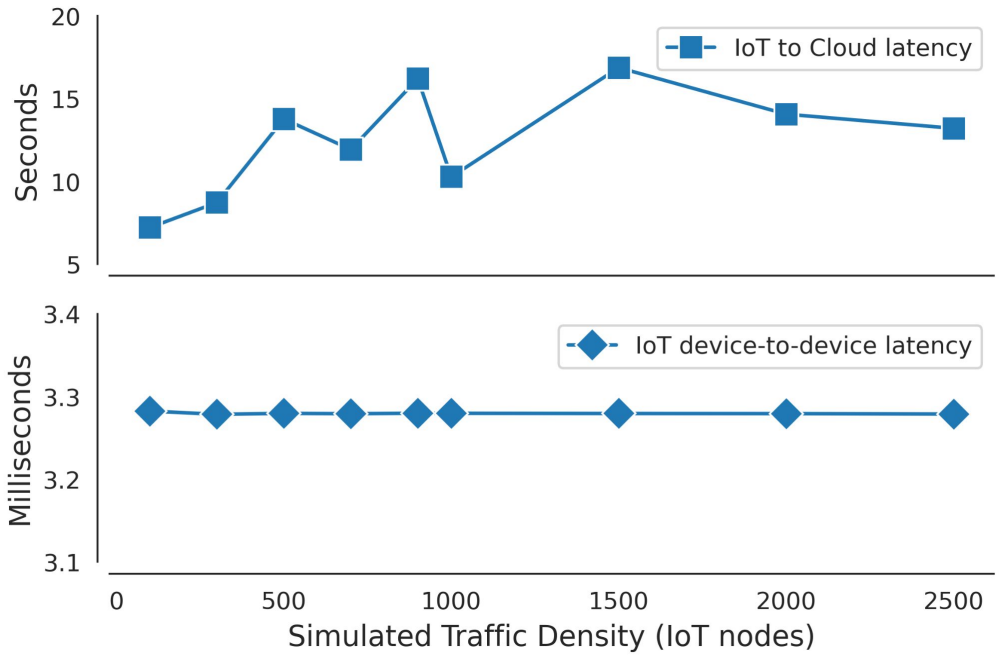


IoT Applications using **Atlas** – Smart Home Automation



IoT Applications using **Atlas** – Smart City Simulation



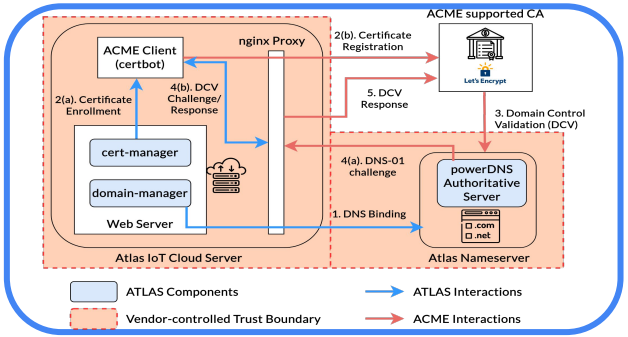


Mobile IoT Nodes (vehicles) interacting with Static IoT Gateways (traffic lights)

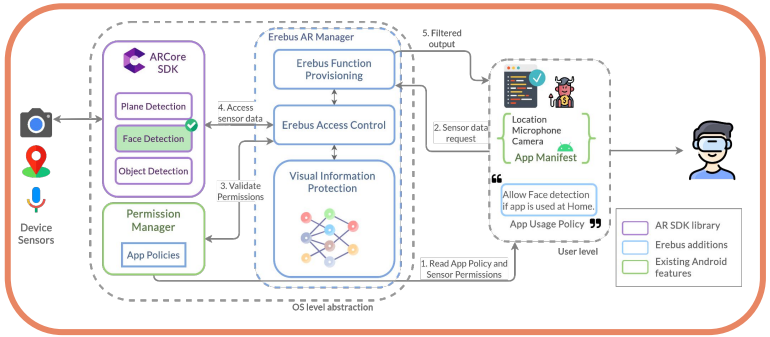
Atlas: Cross-Vendor IoT Device-to-Device Authentication

- Heterogeneity in IoT devices and their supported platforms has led to a fragmented ecosystem.
- We studied existing authentication systems, and identified key trends that inhibit interoperability and scalability between vendors and service providers.
- We developed Atlas to adapt the ACME model of Web PKI for the IoT ecosystem.
- Atlas enables secure and direct device-to-device communication pathways, demonstrating performant and scalable IoT applications.

Securing Mobile Infrastructures against Privacy Attacks



Atlas



Erebus

Sanket Goutam

Stony Brook University